

Branch Prediction in C++: Why Your CPU Is Guessing (And How to Fix It)

High-Performance C++: From Branch Prediction to Branchless Benchmarking



Sagar

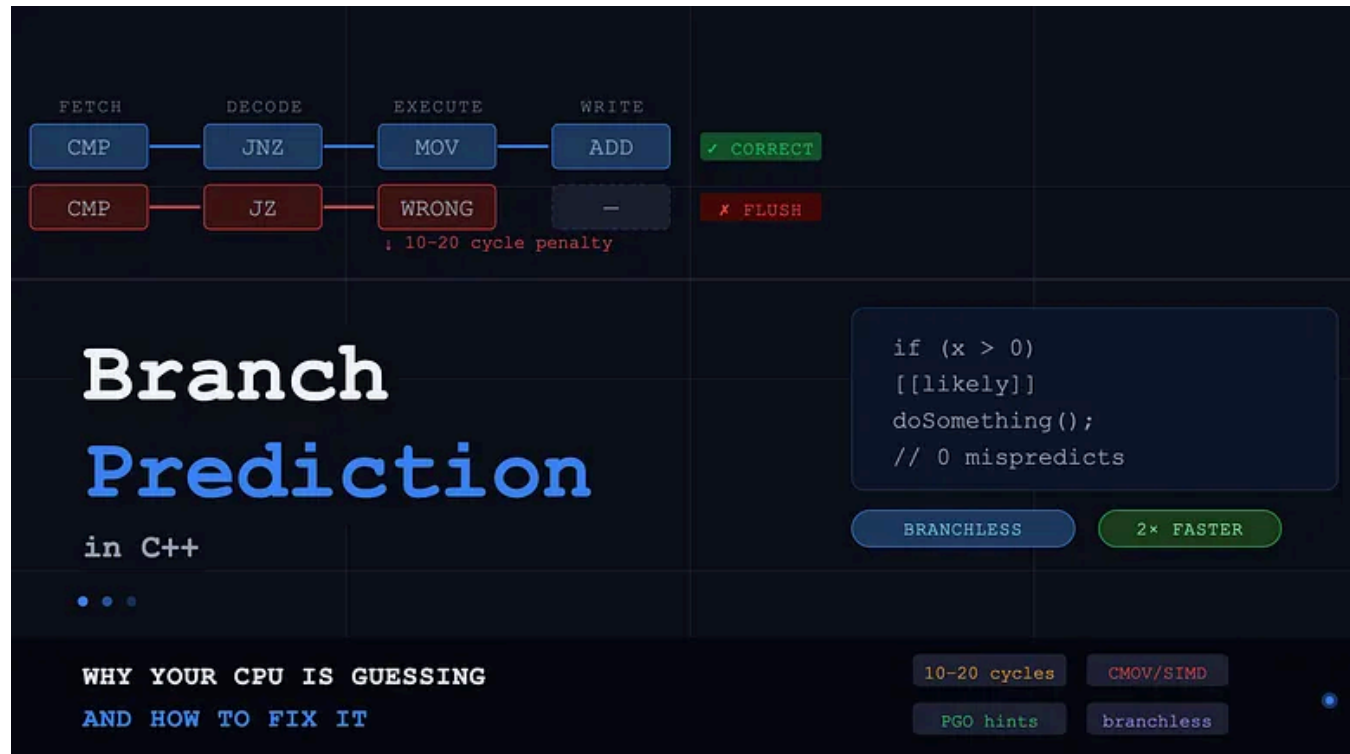
Following ▾

11 min read · 2 days ago



31





What Actually Happens Inside Your CPU?

Modern CPUs don't execute one instruction, wait, then execute the next. They work like an assembly line: while one instruction is being executed, the next is being decoded, the one after that is being fetched from memory, and so on. Five or more instructions deep, all overlapping.

This is called **pipelining**, and it's why your CPU can do billions of operations per second.

Now you write this:

```
if (x > 0) {  
    doSomething();  
} else {  
    doSomethingElse();  
}
```

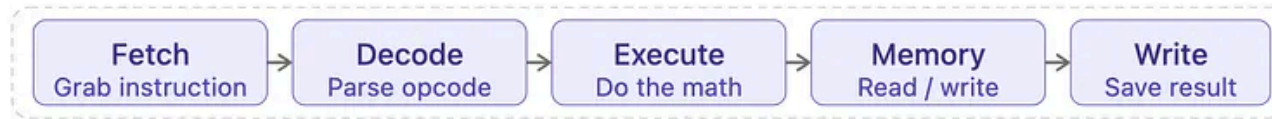
The CPU fetches the `if`. It starts decoding. It starts prefetching the next instruction — but wait. Which instruction? It doesn't know yet. The condition hasn't been evaluated. But stalling the whole pipeline until it knows would cost a dozen cycles.

This is **branch prediction**: the CPU picks a path, speculatively executes it, and hopes it was right. If it was wrong, it has to throw away all that speculative work and start over. That penalty — the pipeline flush — costs roughly **10 to 20 cycles** every time it happens.

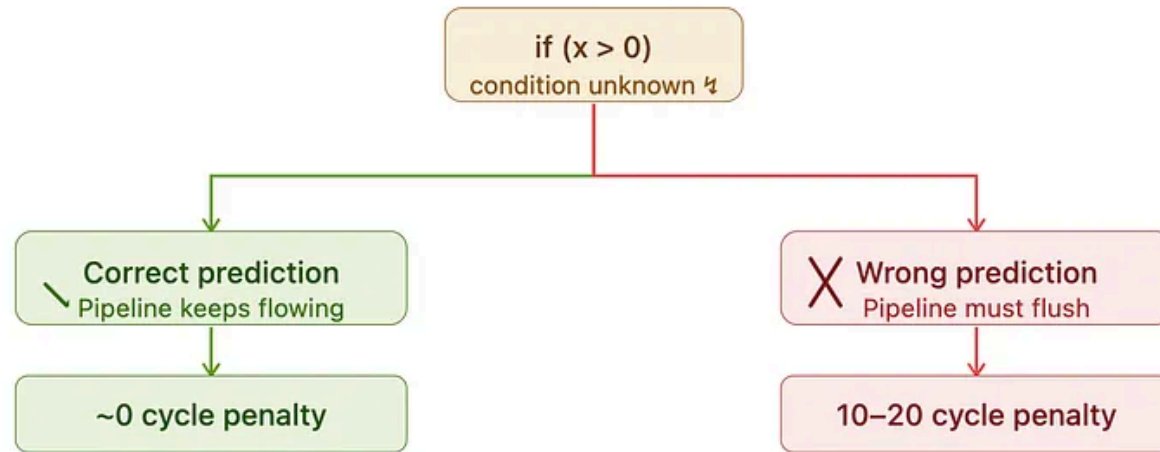
Visually presenting:

The branch prediction dilemma

The pipeline — instructions flow like a conveyor belt



An if statement arrives — the CPU must guess now, evaluate later



On a mispredict — all speculative work is discarded



At 3 GHz, that's about 5 nanoseconds. Doesn't sound like much until you're inside a loop that runs a million times.

What Counts as a Branch?

The obvious ones you already know:

```
if (x > 0) { ... }           // if statements
for (int i = 0; i < n; i++)  // loop conditions
switch (state) { ... }      // multi-way branches
break; continue; return;    // control flow
```

The less obvious ones that will bite you:

```
int val = (a > b) ? a : b;    // Ternary operator(sometimes – depends on t
bool ok = isValid() && isReady(); // Short-circuit evaluation (&&, ||)
```



And the hidden ones most people don't think about:

```
base_ptr->virtualMethod();    // Virtual function call (vtable lookup)
```

```
callback_fn(data);                // Function pointer
```

Every one of these is a point where the CPU has to make a guess. ▶

So what to do? Simple give more information to your compiler so that it can help the CPU to make a better guess.

Strategy 1: Telling the Compiler What You Know

If you know which branch is more likely, you can tell the compiler. In C++20:

```
if (packet.isValid()) [[likely]] {  
    processPacket(packet);        // This is the common case  
} else [[unlikely]] {  
    logError("corrupt packet");   // This rarely happens  
}
```

Before C++20, GCC has a built-in that does the same thing(You'll see this pattern everywhere in Linux kernel code and older C++ codebases):

```
#define likely(expr)    __builtin_expect(!!(expr), 1)
#define unlikely(expr) __builtin_expect(!!(expr), 0)

// Usage
if (likely(connection.isAlive())) {
    sendData();
}
if (unlikely(errno != 0)) {
    handleError();
}
```

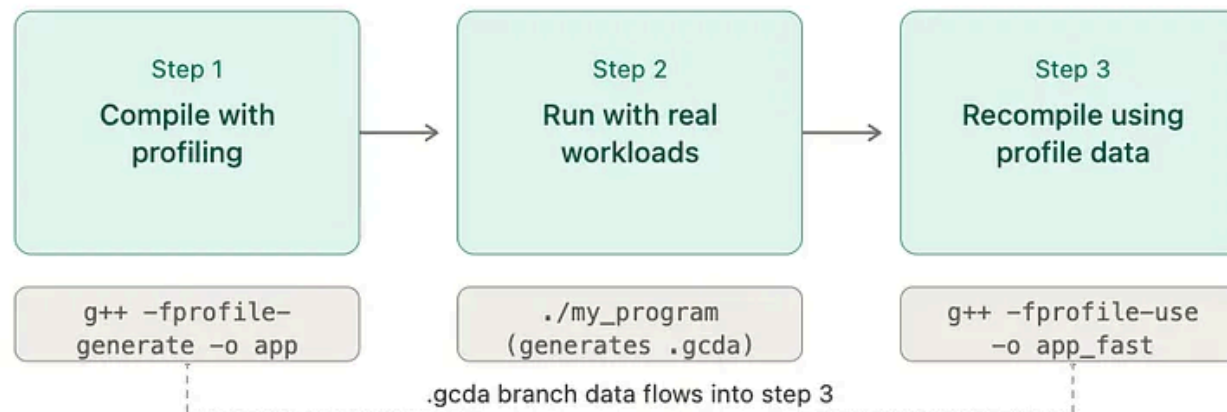
***Remember:** These are **hints**. Always check the generated assembly (`-s` flag or Compiler Explorer) to verify the compiler actually listened.*

The general heuristic — even without explicit hints — is that **the CPU assumes the `if` block is more likely than the `else` block**. So if you have an error case, put it in the `else`. Put your happy path first.

Strategy 2: Profile-Guided Optimization: Let the Data Decide

If you don't know which branches are hot, you can let GCC figure it out automatically.

Profile-Guided Optimization (PGO) is a two-pass compilation process:



```
# Step 1: Build with instrumentation
g++ -O2 -fprofile-generate -o my_app main.cpp

# Step 2: Run representative workloads
./my_app --run-typical-scenario

# Step 3: Rebuild with collected data
g++ -O2 -fprofile-use -o my_app_optimized main.cpp
```

GCC will use the branch frequency data to make smarter optimization decisions — including choosing better branch layouts and potentially eliminating some branches entirely.

The catch: this only works if your test runs actually reflect real usage. If your hottest code path in production is rare during testing, the profiler won't help much there.

Strategy 3: The Harder Fix — Fewer Branches

The best branch is one that doesn't exist. Instead of optimizing how the CPU guesses, just give it less to guess about.

The worst offenders to look for:

- Long `if-else-if` chains
- Deeply nested conditionals
- Branches that depend on user input, random numbers, or system time — anything unpredictable

Some approaches that actually work:

Combine error checks. Instead of checking for five different errors in five different places, collect them and check once at the end.

Unroll loops. If a loop runs a known number of times, the compiler (or you) can unroll it so the branch condition disappears.

Use lookup tables. The classic `islower()` example: instead of comparing character ranges with two comparisons and a short-circuit branch, you precompute a 256-byte table and just index into it. No branches at all.

```
// Slow – has a branch hidden in &&
bool slow_islower(char c) {
    return (c >= 'a') && (c <= 'z');
}

// Fast – branchless table lookup
bool fast_islower(char c) {
    return _islower_table[(unsigned char)c];
}
```

Branchless Programming

This is where it gets fun — and occasionally dangerous.

The idea is to replace conditional logic with arithmetic. Instead of branching, you compute both outcomes and use math to select the right one.

The Ternary Trick:

The CPU has a special instruction called CMOV (conditional move) that performs an assignment without branching. The ternary version gives the compiler a cleaner signal that this is what you want.

```
// Branching version – two possible paths
int max;
if (x > y) {
    max = x;
} else {
    max = y;
}

// Branchless version – compiler can emit a single CMOV
int max = (x > y) ? x : y;
```

Both are semantically identical, but the ternary form makes it trivially easy for the compiler to emit:

```
cmp    eax, ebx    ; compare x and y
cmovl  eax, ebx    ; if x < y, move y into result – NO branch
```

Booleans Are Just 0 and 1:

In C++, true is 1 and false is 0.

Once you internalize this, a whole category of tricks opens up:

```
// Branching
if (inc_flag) {
    x++;
}

// Branchless – the bool IS the increment
x += (int)inc_flag;
```

```
// Branching
if (clear_flag) {
    x = 0;
}
```

```
// Branchless – multiply by the inverse  
x *= (int)!clear_flag;
```

Whether the multiplication version is actually faster depends on your compiler and target architecture. A smart compiler might convert it to a `CMOV` anyway. Always benchmark.



Lookup Tables:

Replace computation *and* branches with a single memory access.

```
// Branching + computation  
#define islower(c) (((c) >= 'a') && ((c) <= 'z'))  
  
// Branchless (single array lookup)  
#define islower(c) _islower_table[(unsigned char)(c)]
```

Real-World Example: Branchless RELU

RELU is a common activation function in neural networks: return x if positive, 0 if negative.

```
// Version 1: Branching
int relu(int x) {
    if (x < 0) return 0;
    return x;
}

// Version 2: Ternary (likely emits CMOV)
int relu(int x) {
    return (x < 0) ? 0 : x;
}

// Version 3: Bitmask
int relu(int x) {
    unsigned int mask = (unsigned)((((int)(x < 0)) - 1));
    return x & mask;
}
```

Input x:	-5	0	7
(x < 0):	1	0	0

subtract 1:	0	0xFFFFFFFF	0xFFFFFFFF	
x & mask:	0	0	7	
RELU(x):	0	0	7	✓

The ternary version is usually the right choice here. It's readable, and it gives the compiler the best chance to emit efficient code.

A Warning About Bitshift Tricks!

You'll often see code like this in low-level C/C++:

```
unsigned int mask = x >> 31; // Sign bit extended to all bits
```

This exploits the fact that right-shifting a signed negative integer fills the vacated bits with 1s on most platforms. But — and this is important — **this is undefined behavior in C++**. The standard doesn't guarantee sign extension on right shift. It works on x86, but if you're writing portable code, don't rely on it.

The safe version loses the branchless benefit:

```
unsigned int mask = (x >= 0) ? 0 : 0xFFFFFFFF;
```

The HFT Paradox

Here's an interesting wrinkle that illustrates the limits of branch prediction theory.

In high-frequency trading, the “hot path” — the actual trade submission — is executed very rarely. The cold path (all the checks that decide *not* to trade) runs almost constantly.

Market Data Tick → Decision Logic → No Trade (99.99%)
└→ EXECUTE TRADE (0.01%) ← HOT PATH!

Branch prediction logic will learn to predict the cold path. It'll be right 99.9% of the time. But the one time you actually want to trade, the CPU is going to mispredict and flush the pipeline — exactly when you need it most.

The usual advice about branch prediction is almost backwards for HFT. You want the CPU to always predict the hot path, even though it'll be wrong most of the time. There's no hardware flag for this. It's one of those places where low-level optimization theory runs headfirst into domain reality.

Advanced Tricks (When You Really Care)

The techniques above cover 90% of branch optimization. But when you're chasing the last few percent — in HFT, game engines, DBMS kernels, or ML inference — there are heavier weapons. These are not everyday optimizations — but on hot paths, they can dominate performance.

Data Partitioning: Make Branches Disappear by Sorting

Instead of branching on every element, reorganize your data so the branch becomes trivially predictable — or unnecessary entirely.

```

// NOT GOOD: Branch on every element — unpredictable
for (auto& order : all_orders) {
    if (order.side == Side::Buy) {
        processBuy(order);
    } else {
        processSell(order);
    }
}

// GOOD: Partition first, then process without branches
auto mid = std::partition(all_orders.begin(), all_orders.end(),
    [](const Order& o) { return o.side == Side::Buy; });

std::for_each(all_orders.begin(), mid, processBuy);    // No branch
std::for_each(mid, all_orders.end(), processSell);    // No branch

```

The upfront `std::partition` cost pays for itself many times over when the processing loop is non-trivial. This pattern also plays beautifully with cache prefetching — you're now streaming through contiguous, homogeneous data.

The same idea applies at a higher level: Structure of Arrays (SoA) instead of Array of Structures (AoS), separate hot fields from cold fields, and keep your branch-free loops iterating over tightly packed, same-type data.

Devirtualization: Killing the Invisible Branch

Every virtual function call is an indirect branch — the CPU loads a pointer from the vtable, jumps to an address it can't predict at compile time. On a tight loop, this is devastating.

```
// BAD: Virtual dispatch – indirect branch on every call
class Handler {
public:
    virtual void process(const Event& e) = 0;
};

void run(Handler* h, const std::vector<Event>& events) {
    for (const auto& e : events) {
        h->process(e);    // vtable lookup every iteration
    }
}
```

Fix 1: CRTP (Curiously Recurring Template Pattern) — compile-time polymorphism:

```
// GOOD: Static dispatch – no vtable, fully inlineable
template <typename Derived>
class Handler {
public:
    void process(const Event& e) {
```

```

        // Compiler knows the exact type – no branch
        static_cast<Derived*>(this)->processImpl(e);
    }
};

class FastHandler : public Handler<FastHandler> {
public:
    void processImpl(const Event& e) { /* ... */ }
};

```

Fix 2: `final` keyword — hints the compiler to devirtualize:

```

// GOOD - Compiler can devirtualize since no further subclasses exist
class FastHandler final : public Handler {
public:
    void process(const Event& e) override { /* ... */ }
};

```

GCC and Clang can both devirtualize `final` classes, turning the indirect call into a direct call (or even inlining it). Use `-fdevirtualize` (on by default with `-O2`).

SIMD: Process 8–16 Elements Without a Single Branch

SIMD (Single Instruction, Multiple Data) replaces scalar branches with parallel mask operations. Instead of asking “is this element above the threshold?” one at a time, you ask about 8 or 16 elements simultaneously — and the answer is a bitmask, not a branch.

```
// Conceptually:  
y = max(x, threshold);
```

The SIMD version is just the vectorized form of this idea.

```
#include <immintrin.h> // AVX2  
  
// Process 8 integers at once – zero branches  
int64_t sum_above_threshold_avx2(  
    const int* data, size_t n, int threshold  
) {  
    __m256i vthresh = _mm256_set1_epi32(threshold - 1);  
    __m256i vsum     = _mm256_setzero_si256();  
  
    for (size_t i = 0; i + 8 <= n; i += 8) {  
        __m256i vdata = _mm256_loadu_si256(  
            (const __m256i*)(data + i)  
        );  
        // Compare: all 1's where data > threshold-1, else 0's
```

```

    __m256i mask = _mm256_cmpgt_epi32(vdata, vthresh);
    // Zero out elements below threshold – no branch!
    __m256i masked = _mm256_and_si256(vdata, mask);
    vsum = _mm256_add_epi32(vsum, masked);
}

// Horizontal sum of the 8 lanes
alignas(32) int tmp[8];
_mm256_store_si256((__m256i*)tmp, vsum);
int64_t total = 0;
for (int i = 0; i < 8; ++i) total += tmp[i];
return total;
}

```

This is conceptually identical to the bitmask trick from earlier — but the CPU does it on 8 values in a single cycle. No branch, no prediction, no miss. Compile with `-mavx2`.

Code Layout: `[[cold]]` and Function Splitting

Error handling code, logging, and rare paths **physically sit next to** your hot code in the binary. This wastes precious instruction cache space. Force the compiler to move cold code far away:

```

// Cold code gets placed in a separate .text section
[[gnu::cold]] void handleCorruptPacket(const Packet& p) {

```

```

        log("Corrupt packet from " + p.source());
        metrics::increment("corrupt_packets");
        // ... heavyweight error handling
    }

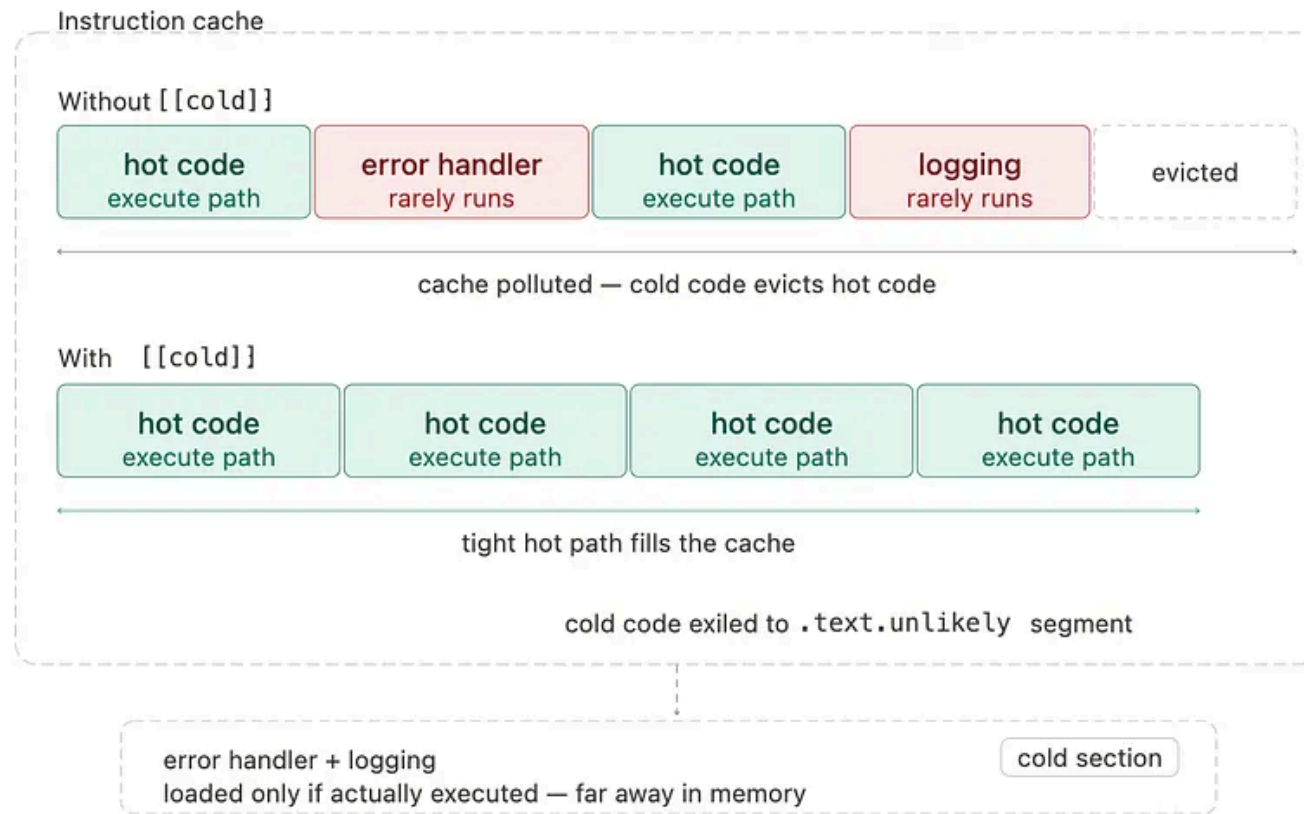
    void processPacket(const Packet& p) {
        if (!p.isValid()) [[unlikely]] {
            handleCorruptPacket(p); // Rare – and now far away in memory
            return;
        }
        // Hot path continues here, tightly packed in icache
        decode(p);
        route(p);
        acknowledge(p);
    }

```

The `[[gnu::cold]]` attribute (or `__attribute__((cold))`) tells GCC/Clang to:

- Move the function to a separate `.text.unlikely` section
- Optimize it for **size** instead of speed
- Keep it out of the instruction cache until actually needed

Combined with `[[unlikely]]`, this keeps your hot path's instruction cache footprint minimal. You can verify with `objdump -d` — cold functions will be at completely different addresses from your hot loop.



The common thread: every advanced technique is about turning a runtime decision into something resolved at compile time, data-layout time, or instruction-level parallelism. The branch predictor is fast. Not having to predict at all is faster.

Comprehensive Benchmark: Proving It With Numbers

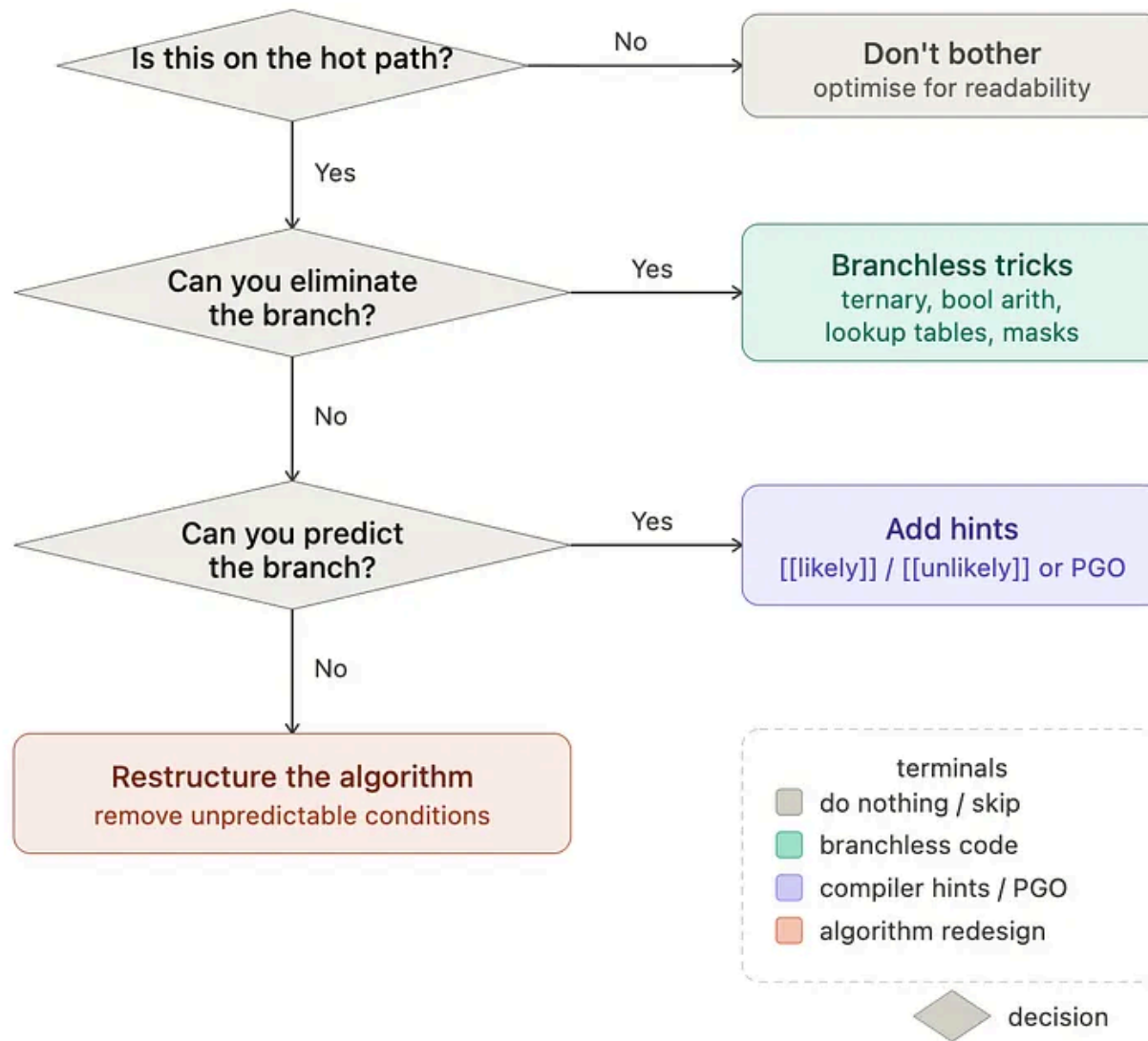
Enough theory. Let's measure everything. The following benchmark tests 7 different techniques across 3 data patterns, and includes integrated `perf` hardware counter collection to show you exactly what the CPU's branch predictor is doing.

You can find the complete benchmark code and analysis here:

Branch Prediction Benchmark

On random data, branchless versions reduced branch-miss rate from ~15% to near zero, cutting execution time by over 2×.

Cheat Sheet: Make your own decision



Closing Thought

In most code, you shouldn't care about any of this. But in hot paths — the **1% of code that runs 99% of the time** — these details are the difference between 'fast' and 'why is this so slow?'

Measure first. Optimize second. And when you do optimize, make the CPU's job as simple as possible.

Found this article helpful?

- **Clap** 🙌 if this changed how you think about branches and performance.
- **Follow** for more deep dives into modern C++ and low-level systems.
- **Drop a comment:** And tell me — what's the most surprising optimization that actually worked for you?

Programming

Software Development

C Plus Plus Language

Cpp

Performance Optimization